Quantifying Process Model Generalization: A Generative N-gram Metric and a Cross-Paradigm Benchmark*

Christian Daniel Krengel and Tianhao Geng

Ludwig-Maximilians-Universität München, Munich, Germany

...

Revision note (delete before submission): blue text is changed or added relative to the previous draft. Removed without trace: the version-lineage table, the two-log version study, and the v1 rows of the D1 tables.

Abstract. Generalization is the ability of a process model to accept behavior that is valid under the underlying process but absent from the event log. It is the least understood of the four established process-model quality dimensions. Published generalization metrics are non-commensurable, are rarely compared against one another and are almost never validated against a ground truth on real-life logs. This report makes four contributions. First, we argue for a *strict construct definition*: a generalization metric should measure only the True-Positive of future valid behavior: measuring False-Positives is the task of precision, and conflating the two destroys the diagnostic value of the quality-dimension quadrant. We use *trace model* and *flower model* as two extreme cases in order to serve as a theoretical test for the metrics and a purity litmus. Second, we present *ShadowGen*, a generative metric that derives a stochastic trace generator from the log using variable-order N-gram statistics with Katz-style backoff and Good-Turing novelty estimation, generates a synthetic shadow log of unseen traces and scores a model by replaying it under two complementary readings (graded fitness and strict acceptance). Third, we design a benchmark which compares our metric and seven published baselines against eight discovery configurations, anchored by variant-based cross-validation as ground truth and assessed on four criteria: correlation, ranking, absolute error, and discriminative spread. Fourth, we evaluate on five real-life event logs spanning 15 thousand to 2.5 million events and differing representativeness, under a uniform compute budget of one hour per cell. ShadowGen tracks the ground truth on every log (Pearson 0.986–0.999, mean absolute error 0.006–0.043) at a median of 15 seconds per model, whereas the field's default metric (PM4Py) correlates negatively with the ground truth on 4 of the 5 logs and fails the flower-model litmus on all 5. Two published metrics return no score on any log within budget. An adaptation of the published bootstrap generalization reaches the same calibration at 2.5× the runtime, which corroborates the generative paradigm beyond our specific generator. The strict acceptance reading, validated against an

* Report, M.Sc. Computer Science, LMU Munich.

acceptance-based ground truth, anchors both construct poles exactly on every log. A direct test of the generator premise shows that up to 26% of the generated traces are exact matches of real held-out variants.

Keywords: Process mining · Process discovery · Generalization · Model quality · Conformance checking · Benchmark.

1 Introduction

1.1 Problem statement and motivation

Process discovery algorithms construct process models from event logs recorded by information systems [1]. A discovered model is conventionally assessed along four quality dimensions [7]: *fitness* (the model allows observed valid behavior), *precision* (it does not allow invalid behaviors), *simplicity* (it is structurally sparse) and *generalization* (it allows future valid behavior). An event log is only a sample of an underlying process. Generalization captures whether a model describes the process rather than memorizing the sample.

Generalization is the most elusive of the four dimensions for a simple reason: it attempts to make a claim about behavior that, by definition, is not in the data. The literature offers various metrics, but they approach the problem from fundamentally different angles: counting rarely used model parts [7], information-theoretic relevance of stochastic models [4], adversarial anti-alignments [8], generative adversarial networks [22], statistical bootstrapping [18] and biodiversity-inspired species estimation [12]. These different paradigms produce different scores that are not clearly commensurable. Prior work has compared such metrics to one another and found that, unlike fitness and precision, generalization measures do not agree [11], and has assessed conformance measures against axiomatic propositions [2,21]. What we provide is a comparison across the various paradigms validated against an empirical, log-derived generalization ground truth (variant-based k-fold) on common real-life logs, which alone can establish which metric, if any, tracks unseen-but-real behavior. The practical consequence: no validated consensus generalization metric exists. The de-facto default, the generalization measure shipped in PM4Py [6], is widely used but, as we show, does not track empirical generalization. A practitioner who wants to know whether a discovered model will accept tomorrow's cases is therefore left without a validated, affordable instrument.

1.2 Proposed solution

We pursue two goals. The first is a *new metric*. *ShadowGen*[1] learns a variable-order Markov model of the log: N-grams up to order N=6 with Katz-style back-off [14]. From this, Good-Turing estimation [9] gives a per-context probability

[1] The implementation package keeps the legacy name `HybridGen`, from an earlier design that paired the generative score with a structural term. That term was retired (Sect. 4.2), so the metric here is the pure generative component.

that the next event is new. ShadowGen then generates a synthetic *shadow log* of plausible-but-unseen traces and scores a model by how well it replays them [20]. The second goal is a *validated benchmark*. We test the metric and seven published baselines on eight discovery configurations and five real-life logs, and validate each metric against variant-based hold-out (both k-fold cross-validation and leave-one-variant-out) which measures acceptance of unseen-but-real behavior directly.

1.3 Summary of key results

ShadowGen reproduces the cross-validation ranking and is calibrated to it on every log (Pearson 0.986–0.999, mean absolute error 0.006–0.043) at a median of 15 seconds per model. The same agreement also holds under leave-one-variant-out hold-out.

An adaptation of the published bootstrap generalization is the only baseline of comparable quality: two independently designed synthetic-log generators, scored the same way, land on the same calibration (mean absolute error 0.018 against our 0.022, averaged over the logs), at 2.5× the runtime. We read this as corroboration of the generative paradigm rather than competition (Sect. 6.5). The widely used PM4Py metric tracks the ground truth on no log: it correlates negatively on 4 of 5 and fails the flower-model litmus on all 5. Two published metrics return no score on any log and are reported as infeasible. A strict acceptance reading of the same shadow log, validated against an acceptance-based ground truth (Pearson 0.992–0.999 where defined), anchors the construct's two poles at exactly 0 and 1 on every log and exposes a depth limit of strict replay on 57-event traces. A direct premise test shows that up to 25.9% of generated traces are exact matches of held-out real variants, against 0% for random traces.

1.4 Positioning within process mining

This work sits between conformance checking and model-quality evaluation. Most prior generalization metrics propose a measure and demonstrate it qualitatively. Our stance is instead validation: a single-shot score is trustworthy only if it agrees with a direct, expensive measurement of generalization (hold-out acceptance). We also adopt a deliberately narrow definition of generalization: focusing only on the acceptance/fitness of future valid behavior, separate from precision. This allows us to turn the otherwise-useless flower model into a diagnostic litmus test. Finally, any metric and any hold-out ground truth is computed from a log, so its accuracy is bounded by how representatively the log samples the system [13]. This bound applies to every log-based analysis, not just ours, so we treat representativeness as a property of the data and span it deliberately across our datasets. Detailed positioning is deferred to Sect. 3.

1.5 Scope

We evaluate on five real-life logs, D1–D5, with the same battery of methods on each, under a uniform one-hour compute budget per cell (Sect. 6.1). The result

matrix is complete: every cell holds either a measured score or an evidence-bearing sentinel. Some external baselines cannot be run as published. We either adapt them to a shared protocol or report them as infeasible, and we state every such case explicitly (Sects. 5.4, 6.2).

1.6 Structure

Section 2 fixes notation and the process-mining concepts the method builds on. Section 3 surveys related metrics by theme and states the gap. Section 4 presents the construct, the metric, its pseudocode, a running example, and a complexity analysis. Section 5 describes the implementation and where it deviates from the formal method. Section 6 reports the benchmark: setup, feasibility, the cross-paradigm comparison, results at scale, the acceptance reading, the generator premise, runtime, discussion, and threats. Section 7 concludes.

2 Background

Event logs. Let $\mathcal{A}$ be a finite set of activities. A trace $\sigma = \langle a_1, \ldots, a_k \rangle \in \mathcal{A}^*$ is a finite sequence of activities; an event log L is a multiset of traces. Traces with an identical activity sequence form a *variant*; we write $V(L)$ for the set of variants and, for a variant $v \in V(L)$, $\#v$ for its case count. Real logs are typically dominated by a few frequent variants and a long tail of rare ones. We summarize a log's repetitiveness by $\mathrm{TLRA} = 1 - |V(L)|/|L|$, the empirical probability that a new case repeats a known variant.

Process models and discovery. A process discovery algorithm (a miner) maps a log to a process model. In this work all models are accepting Petri nets (N, m_i, m_f) discovered with PM4Py [6]; the discovery algorithms we use range from the Alpha miner [3] through the Heuristics miner [23] to the Inductive miner [15,16]. Two extreme constructions serve as theoretical boundaries: a *trace model*, one isolated path per observed variant, and a *flower model*, all activities in a single self-loop accepting every sequence over $\mathcal{A}$.

Token replay and its two readings. Token-based replay [20] replays a trace on a net, firing transitions and accounting for produced, consumed, missing, and remaining tokens. It yields two distinct quantities that this report keeps separate:

- a graded trace fitness in $[0, 1]$ derived from the token bookkeeping, which awards partial credit (a trace the net cannot actually execute end-to-end can still score, e.g., 0.7);
- a boolean perfect-replay flag (`trace_is_fit`): the trace replays with no missing or remaining tokens, i.e., it is in the model's language.

The fitness of a log is the mean trace fitness; the acceptance rate is the fraction of perfectly replayed traces. As Sect. 6.6 shows, the graded and boolean readings can diverge sharply, and the gap between them is itself informative.

Variable-order language models. The generator at the core of our metric is a variable-order Markov (N-gram) model of the log. Two classical techniques from statistical language modeling are reused. *Katz backoff* [14] predicts the next symbol from the longest recent context that has sufficient statistical support, falling back to shorter contexts when a long context is too sparse to trust. *Good–Turing estimation* [9] estimates the total probability mass of unseen events from the number of events observed exactly once (singletons): if a fraction N_1/N of observations were one-offs, roughly that fraction of future observations should be expected to be novel.

3 Related Work

We group existing approaches by paradigm. Method identifiers (M2–M8) anticipate the benchmark roles of Sect. 6.

Structural counting (M2). The PM4Py built-in generalization metric [7,6] penalizes models whose internal parts are visited rarely while replaying the observed log: a model in which every node is used often is deemed general. It is computationally trivial but never confronts the model with unseen behavior, and, as our results show, it does not track held-out acceptance.

Entropy-based (M3). Entropic relevance [4] measures the average number of bits needed to encode the log under a *stochastic* model (a stochastic deterministic finite automaton). It unifies fitness- and precision-like aspects in one information-theoretic quantity, but requires a stochastic model; a plain Petri net must first be annotated with transition probabilities, which is a non-trivial conversion.

Adversarial (M4). Anti-alignments [8] search for model traces maximally distant from the log: a model able to produce behavior far from everything observed is judged over-general. The underlying optimization is exponential in nature, which makes the published implementation infeasible on real-life logs (Sect. 6.2).

Generative / neural (M5). AVATAR [22] trains a sequence GAN to approximate the system that produced the log, samples unseen-but-plausible traces from the generator, and scores generalization as the harmonic mean of fitness and precision against those samples. AVATAR is conceptually closest to our metric (both are generative) but replaces an interpretable statistical generator with a neural one at a cost of hours of GPU training per log, and its harmonic-mean score is an F-measure that blends precision into the result (see the construct discussion in Sect. 4.1).

Resampling / bootstrap (M6adapted, M6original). Bootstrap generalization [18] treats the log as a sample from an unknown trace population, repeatedly resamples it and "breeds" recombined traces via genetic crossover, and aggregates model quality over the replicates with confidence intervals. Its target quantity is model–system recall (the fraction of system behavior the model accepts), which coincides with our construct (Sect. 4.1). It is the strongest baseline in our study. Crucially, its test traces are recombinations of observed material; it does not model or inject genuinely novel events in a principled way.

Species / diversity (M7). SpeciAL [12] transfers species-richness estimation from ecology to event data: activities or N-grams are "species," and completeness/coverage profiles quantify how representative a log (or a simulated log) is. We use the coverage ratio between model-simulated and original logs as a generalization proxy.

Pattern-based (M8). Reißner et al. [19] read generalization off automata-based behavioral patterns; the available implementation proved unusable on our logs (Sect. 6.2).

Hold-out as empirical ground truth. The most direct operationalization of generalization is hold-out evaluation: discover a model on a subset of variants, then measure how well withheld variants replay. Variant-based k-fold cross-validation and leave-one-variant-out realize this at increasing granularity. They are expensive (leave-one-variant-out requires one discovery per variant) and, importantly, are only defined when the model can be rediscovered from log subsets, so they evaluate a discovery algorithm rather than a model artifact in hand. These properties make them ideal as a validation reference but impractical as a routine metric.

Membership versus graded semantics. The textbook definitions of generalization are membership-shaped: "the likelihood that previously unseen but valid behavior is allowed by the model" [1], a yes/no question per trace. The prevailing practice, by contrast, is graded, because token- and alignment-based fitness were introduced precisely so that an almost-correct trace does not count the same as noise [20]. Our metric reports both readings of the same shadow log and validates each against its matching ground truth.

Comparative and axiomatic evaluation. A few studies assess the metrics themselves rather than models. Janssenswillen et al. [11] ran a large-scale comparison of fitness, precision, and generalization metrics and found that, unlike the other two dimensions, generalization metrics *do not agree with one another*. A follow-up tested whether such measures capture model–system quality on synthetic logs with a known ground-truth system, and concluded that it is "nearly

impossible to objectively measure the ability of a model to represent the system" [10]. A second strand evaluates conformance measures against *axioms*: van der Aalst's conformance propositions [2] and Syring et al.'s check of which measures satisfy them [21]. A separate line benchmarks process *discovery algorithms* using these metrics as given [5], not the metrics themselves. None of these compares a cross-paradigm set of generalization metrics against an empirical hold-out reference on real logs.

The gap we fill. Three gaps recur across these groups. (i) *No cross-paradigm validation.* To our knowledge, no prior work compares generalization metrics across paradigms against an empirical, log-derived hold-out ground truth on common real logs; the closest attempts compare metrics only to one another [11] or use synthetic known-system data [10]. It is therefore unknown which paradigm actually tracks held-out acceptance. (ii) *Opacity.* The most faithful baselines (neural, bootstrap) return a number with little insight into why a model scored as it did. (iii) *Construct drift.* Several "generalization" metrics quietly fold in precision, which conflates two orthogonal failure modes. Our contribution is a metric that is (a) validated against cross-validation ground truth and shown to track it, (b) interpretable, exposing per-context novelty, mutation flags, an openness profile, and a strict acceptance reading, and (c) construct-pure by design, with a benchmark that makes construct drift visible through the flower-model litmus.

4 Method

4.1 Construct: what generalization is and is not

We adopt a strict construct: **generalization is the degree to which a model accepts future, valid behavior of the underlying process that is absent from the log, and nothing else.** Generalization is *not* precision. Detecting over-permissiveness (acceptance of *invalid* behavior) is the precision dimension's task. Folding a permissiveness penalty into a generalization score makes any mid-range value ambiguous: one cannot tell "rejects future valid behavior" from "accepts invalid behavior." That destroys the diagnostic value of the quality quadrant. One does not redefine recall because a degenerate classifier attains maximal recall; one reports precision next to it. Metrics that blend the two (for example, AVATAR's harmonic mean of fitness and precision) measure a legitimate but *different* construct and must not be read as pure generalization.

Two degenerate models operationalize the poles. The *flower model* accepts every sequence over $\mathcal{A}$; its generalization is maximal *by definition*, and it is a useless model because of its precision and simplicity, not its generalization. A pure generalization metric must therefore score it ≈ 1. The *trace model* memorizes observed variants and accepts nothing unseen; it is the 0 pole. This yields a *construct-purity litmus*: a metric that scores the flower model below 1 demonstrably mixes precision or structure into its score. The metric is meant to be read

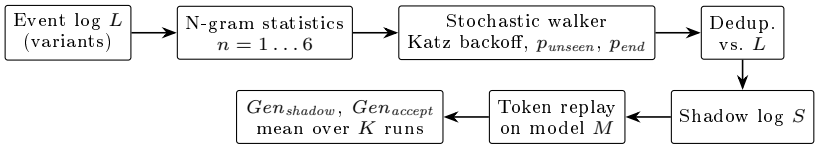

Fig. 1. ShadowGen pipeline. The generator is derived from the log only; the model under evaluation enters only at the replay step.

alongside precision: a model with $Gen \approx 1$ and precision ≈ 0 is diagnosed as over-permissive by the precision column, exactly where that information belongs.

4.2 Framework

ShadowGen scores a model by the replay quality of a synthetic *shadow log*: traces that are plausible under the log's behavior but, by construction, absent from it. The generator is derived from the log alone, and the model under evaluation is consulted only at replay time (Fig. 1). This keeps the generation independent of the artifact being judged. An earlier design paired this generative score with a structural term that penalized rarely-used model parts; we dropped it, because its anti-flower role belongs to precision (Sect. 4.1) and its anti-overfitting role is already covered by replay failures of recombined traces. The current score is therefore the pure generative component.

4.3 Shadow log generation

Variant statistics. The log is compressed to its variants with case counts; every count below is weighted by how often the variant occurs ($\#v$). For each order $n \in \{1, \ldots, N\}$ (default N=6) and each observed context s (a length-n activity sequence), we record the successor multiset $C_n(s)$, where $C_n(s)[a]$ is the frequency with which activity a follows s, and separately the termination counts: $T_n(s)$, the number of occurrences of s, and $E_n(s)$, the number of those occurrences at the end of a trace.

Katz backoff. At generation time the walker holds a partial trace σ and resolves its decision context to the deepest order with adequate support. Let τ be a support threshold (default 5). The *resolved order* is the largest $n \leq N$ such that the suffix $s = \text{suffix}_n(\sigma)$ satisfies $|C_n(s)|_1 := \sum_a C_n(s)[a] \geq \tau$ *and* the Good–Turing mass below is < 1; otherwise n is reduced, falling back ultimately to n=1. Thus N is an upper bound, not a fixed operating point: on sparse logs the effective order is reduced naturally.

Good–Turing novelty. For the resolved context s, the probability that the next activity is one never observed after s is estimated as

$$p_{unseen}(s) \;=\; \frac{N_1(s)}{|C(s)|_1}, \qquad N_1(s) = |\{a : C(s)[a] = 1\}|, \tag{1}$$

i.e., the number of singleton successors over the total successor count. With probability $p_{unseen}(s)$ the walker *mutates*; otherwise it *exploits*.

Mutation proposal (Katz-consistent). When a mutation fires, the inserted activity must be novel in the resolved context yet plausible. We draw it from the deepest *shorter* context that offers a successor unseen at the resolved order. Let R be the resolved order and $seen = \{a : C_R(s)[a] > 0\}$. We scan $m = \min(|\sigma|, N), \ldots, 1$ and take the first m for which $\{a : C_m(\text{suffix}_m(\sigma))[a] > 0\} \setminus seen \neq \emptyset$, sampling from that residual set; if none exists we fall back to global activity frequencies restricted to $\mathcal{A} \setminus seen$. A short observation fixes the search direction: a longer context's successor set is always contained in its suffix's, since every occurrence of s is also an occurrence of its suffix s' with the same next activity. Contexts of order $\geq R$ therefore add nothing new (orders $> R$ are subsets of the resolved successors, and order R equals *seen*), so the proposal always finds its candidates at order $R-1$ or shorter, one level below the resolved order. An earlier design drew the mutated activity uniformly from $\mathcal{A}$ and thus injected behavior that was often process-impossible at that position; the Katz-consistent proposal replaces it.

Exploitation: two sampling modes. For a non-mutation step the walker samples a seen successor a with weight $w(C(s)[a])$, normalized over the successor set. We support two weighting modes:

$$w_{\text{mle}}(c) = c \qquad \text{vs.} \qquad w_{\log}(c) = \ln(c+1). \tag{2}$$

The `mle` mode samples from the maximum-likelihood estimate of the next-activity distribution (proportional to observed frequency), so the shadow log mirrors the *expected* future. The `log` mode deliberately flattens the distribution, up-weighting rare paths; it is a rare-behavior *stress test*. The `mle` mode is the default and is what the evaluation reports; the calibration case for it is measured in Sect. 6.5. The same weighting governs the mutation-proposal sampling, but never the mutation *rate* of Eq. (1), which always uses raw counts.

Context-aware termination. Whether the walk ends after context s is decided by a termination probability resolved with the same backoff:

$$p_{end}(s) \;=\; \frac{E(s)}{T(s)}. \tag{3}$$

Conditioning termination on the n-gram context (rather than the last activity alone) fixes premature termination when activities that legitimately end traces appear early in a generated trace.

Deduplication and scoring. Each generated trace is compared against the set of original variants and regenerated if it is a copy (up to a retry cap), so the

Algorithm 1 Shadow-log generation (one iteration)

Require: log L, order N, threshold τ, count T, cap M, weighting w

build C_n, T_n, E_n for $n = 1..N$ from variants of L; collect starts, alphabet, variant set $\mathcal{V}$
$S \leftarrow \emptyset$
for $i = 1$ to T **do**
  repeat
    $\sigma \leftarrow \langle$ sample start activity $\rangle$; $mut \leftarrow$ false
    while $|\sigma| < M$ **do**
      resolve context s by Katz backoff (τ); compute $p_{unseen}(s), p_{end}(s)$
      if $rand() < p_{end}(s)$ **then break**
      end if
      if $rand() < p_{unseen}(s)$ **and** a novel candidate exists **then**
        append a Katz-consistent novel activity (weight w); $mut \leftarrow$ true
      else
        append a seen successor sampled $\propto w(\cdot)$; **break** if dead end
      end if
    end while
  until $\sigma \notin \mathcal{V}$ **or** retry cap reached
  $S \leftarrow S \cup \{(\sigma, mut)\}$
end for
return S

shadow log contains only unseen sequences. A shadow log S of $\min(1000, |L|)$ traces is replayed on the model; generation and replay are repeated K=5 times (all randomness seeded). We report two scores: Gen_{shadow} (mean graded trace fitness, the headline) and Gen_{accept} (fraction of perfectly replayed traces, the strict reading). We also report each score separately for shadow traces that contain a mutation and those that do not, which shows how a model handles genuinely novel events versus mere recombinations.

4.4 Algorithm

Algorithm 1 gives the generation procedure; statistics gathering and scoring are described above.

4.5 Running example

Figure 2 traces one mutation on a synthetic Sepsis pathway (seed 42). Backoff does two jobs. It leaves the over-sparse order-6 context (seen five times, every successor once, so $p_{unseen} = 1$) for order 5 to set *how often* to invent an event ($p_{unseen} = 1/35 \approx 3\%$); when the draw fires, it keeps backing off to order 4 to choose *what* to invent (Release-D). The inserted activity is novel in the exact context, yet attested in a related shorter one, and the resulting trace (absent from the log) is flagged as a mutated shadow trace.

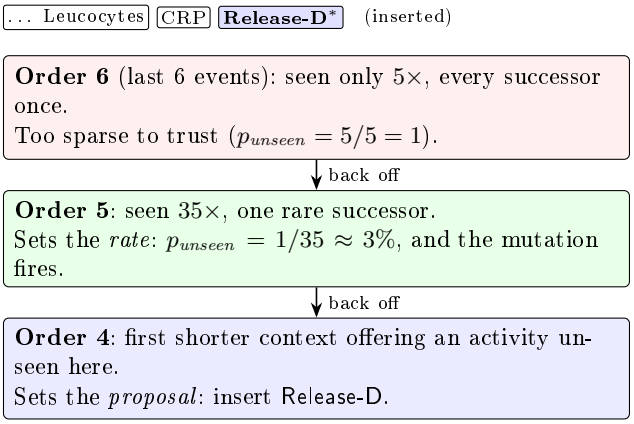

Fig. 2. Katz-consistent mutation (Sect. 4.5). Backoff sets both *how often* to invent an event (the rate, resolved at order 5) and *what* to invent (the proposal, found at order 4): novel in the exact context, yet attested in a related shorter one.

4.6 Time and space complexity

Let $|V|$ be the number of variants, $\bar{\ell}$ the mean variant length, $\ell_{\max}$ the longest, $A = |\mathcal{A}|$, N the maximal order, T the shadow-log size, $M \approx 2\ell_{\max}$ the walk cap, K the iteration count, and $|M_P|$ the size of the Petri net under evaluation. Let $E_V = \sum_{v \in V(L)} |v|$ be the total length of distinct variants.

Statistics ($O(N\,E_V)$). Each of E_V positions contributes to N orders, each $O(1)$. Space is $O(N\,D)$ where $D \leq N\,E_V$ is the number of distinct n-grams stored.

Generation ($O(T\,M\,\bar{b})$, on average). Each of T traces takes up to M steps. Per step, context resolution and termination are memoized on the last-N activities, so each distinct context is resolved once at cost $O(N + b + A)$ (backoff scan, building the successor and novel-candidate lists, b the branching factor), and reused thereafter; sampling a successor costs $O(\bar{b})$ with $\bar{b} \leq A$. Deduplication is an $O(1)$ expected set lookup per attempt. Thus generation is $O(T\,M\,\bar{b}) + O(D\,(N + A))$, i.e., effectively linear in the shadow-log size.

Replay ($O(T\,M\,|M_P|)$). Token replay is roughly linear per trace in trace length and net size.

Total. Per evaluated model, $O\big(K\,(N\,E_V + T\,M\,\bar{b} + T\,M\,|M_P|)\big)$. Two observations matter. First, the cost is linear in the log *through its variants*, not its cases: variant compression makes the metric scale with behavioral diversity rather than volume. Second, this contrasts sharply with the leave-one-variant-out ground truth, which performs $|V|$ full model discoveries ($O(|V| \cdot \text{discover})$) and is therefore orders of magnitude more expensive on variant-rich logs. In the current implementation the statistics are rebuilt once per iteration, contributing the factor K to the $N\,E_V$ term; this is an implementation artifact (Sect. 5.5)

rather than an intrinsic cost, as the model could be built once and reused across iterations.

4.7 Assumptions, scope, and limitations of the approach

The method assumes a control-flow view (activity sequences; data, resources, and time are ignored) and a finite activity alphabet. It assumes the log is a representative-enough sample that local N-gram statistics are meaningful; on extremely sparse logs the backoff collapses toward n=1, reducing the metric to a 1-gram directly-follows generator, which remains a reasonable if weaker baseline (measured as the ablation in Sect. 6.5). The shadow log models future behavior as recombinations of observed local patterns plus a calibrated tail of novel events; behavior driven by long-range or data-dependent dependencies beyond order N is not captured. The metric exposes parameters (N=6, τ=5, K=5), but they are fixed defaults: N and τ are calibrated once (Sect. 6.1) and never tuned per dataset, and Katz backoff makes N an upper bound that adapts downward on sparse logs. In use the metric is therefore as parameter-free as fitness and precision are. Finally, it measures only the generalization construct of Sect. 4.1 and is meant to be read alongside a precision metric, not in isolation.

5 Implementation

5.1 Architecture

The metric is implemented in Python on top of PM4Py [6]. The code is organized as an *algorithm registry*: each metric version registers itself under a name (`v1`, ..., `v2.6`), and a benchmark runner loads a version by name and calls a uniform `evaluate_miner(log, miner)` entry point. Each version is a *frozen module*. Rather than mutating one file over time, every design stage is preserved as the exact code that produced its benchmark numbers, which pins provenance: every reported number can be regenerated from the exact code that produced it, and the 1-gram ablation of Sect. 6.5 is simply an earlier frozen module. The benchmark layer comprises model preparation, per-method bridges (including subprocess bridges to external Java/Docker baselines), the ground-truth scripts, and two runners (`run_m1_family.py` for the metric family, `version_comparison.py` for quick multi-seed checks).

5.2 Key design decisions

Variant compression. Statistics and deduplication operate on variants weighted by case count, making preprocessing independent of case volume (Sect. 4.6). **Memoization.** Context resolutions are cached on the last-N activities, turning the per-step cost effectively constant after warm-up. **Iterative walker.** Generation uses an explicit loop rather than recursion, avoiding stack limits on long walks. **Determinism.** All random number generators are seeded

(*seed*=42) so that every cell is reproducible; the residual nondeterminism of the replay library itself is measured in Sect. 6.10. **Provenance.** Every (dataset, miner, method) cell writes a sidecar JSON capturing exact parameters, raw per-iteration scores, runtime, and integrity counters; these files are the single source of truth from which all tables are derived, and a result without a matching config is treated as invalid. **Integrity counters.** The generator reports `duplicates_kept` (deduplication retry cap exhausted) and `truncated_traces` (walks cut at the length cap), so any deviation of the shadow log from its specification is visible rather than silent.

5.3 The tool in use

The benchmark is driven from the command line. Listing 1.1 shows a run of the metric on one dataset; Listing 1.2 shows a representative result record; Listing 1.3 shows a generated shadow trace with its mutation point (the running example of Sect. 4.5). The tool prints the legacy package name `HybridGen`.

Listing 1.1. Running the metric over all miners on D1.

```
$ python benchmark/run_m1_family.py --dataset D1
Discovering models (cached across methods):
  Trace_Filtered  751t/377p  ...   Flower 16t/1p
--- M1g: HybridGen v2.6 (MLE weighting) ---
    Inductive_Strict   0.9838 +- 0.0020   (6.7s)
    Flower             1.0000 +- 0.0000   (3.5s)
SUMMARY  Pearson 0.996  Spearman 1.000  MAE 0.023  Flower 1.000
```

Listing 1.2. Sidecar config JSON for one cell (abridged).

```
{ "dataset": "Sepsis", "miner": "Inductive_Strict", "method": "M1g",
  "seed": 42,
  "parameters": { "algorithm_version": "v2.6", "max_n": 6,
                  "successor_weighting": "mle", "num_shadow_traces": 1000,
                  "iterations": 5 },
  "results": { "mean": 0.9838, "std": 0.0020, "gen_accept": 0.7444,
               "duplicates_kept": 0, "truncated_traces": 0,
               "runtime_s": 6.7 } }
```

Listing 1.3. A generated shadow trace; the starred event is the Katz-consistent mutation.

```
ER-Reg  ER-Triage  ER-Sepsis-Triage  LacticAcid  CRP  Leucocytes
IV-Antibiotics  Admission  Admission  Leucocytes  CRP  [Release-D]*  Return-ER
```

5.4 Baseline integration realities

Integrating the seven external baselines surfaced engineering realities that materially affect the evaluation and must be reported as such. The PM4Py metric (M2) is a one-line library call. AVATAR (M5) runs in a TensorFlow 1 Docker image and required a greedy longest-match decoder to reconstruct multi-word activity names from the GAN's token output. SpeciAL (M7) is a Python package used directly; on D5 we use the released PyPI package (`special4pm` 0.1.3),

on D1–D4 the source copy provided by the authors' repository, a version skew we disclose (Sect. 6.10). Two baselines could not be used as published: anti-alignments (M4) and pattern-based generalization (M8) did not complete on real logs (Sect. 6.2). A third, entropic relevance (M3), requires a Petri-net-to-stochastic-automaton conversion that is not yet implemented; we currently approximate it with a per-miner DFG simulation (PM4Py `play_out`, 5 000 traces) converted to a probability automaton via the open-source `relevance.jar` (JDFG2Aut + Relevance). The pipeline yields model-discriminating scores on all five datasets, though the transition probabilities are estimated from the simulated DFG frequencies rather than from a proper stochastic analysis of the original Petri net.

Bootstrap (M6adapted) is a construct-faithful adaptation. Polyvyanyy et al. define bootstrap generalization as model–system *recall*: the score is 1 exactly when the model accepts every new trace the system produces [18], the same acceptance construct we use (Sect. 4.1). Our bridge runs the genuine `bsgen` bootstrap-with-breeding *sampler* and scores the bred traces by PM4Py token-replay fitness, i.e. a *graded* model–system recall, the way Gen_{shadow} grades the shadow log. The sampler itself is our reimplementation of the paper's published algorithms (the original source is not public); it reproduces the published D1 values within 0.008 on every cell. We deliberately avoid the Entropia `-bgen` tool's harmonic mean of model–system precision and recall: that F-measure folds precision back in and is no longer the paper's recall-based generalization. It scores the flower model 0.18 instead of 1.0 on D1, failing the litmus of Sect. 4.1, and we carry it through the benchmark as its own method id (`M6original`) so the contamination stays measurable at scale (Sect. 6.5). The price of our choice is twofold. First, M6adapted as we report it is not the off-the-shelf Entropia `-bgen` output but our own token-replay scoring of its genuine `bsgen` sampler, so it is an adaptation, not a drop-in published baseline. Second, that scoring is the conformance core of our metric and of R1, a shared-ruler effect we flag in Sect. 6.10.

5.5 Deviations from the formal method

Three deviations are worth recording. (i) As noted in Sect. 4.6, the N-gram model is rebuilt each of the K iterations rather than once; this is a performance artifact, not a semantic one. (ii) The strict reading counts a shadow trace as accepted when PM4Py's token replay flags it as perfectly fitting (`trace_is_fit`). That flag is a fast but approximate test of language membership, which is properly decided by a cost-zero alignment. On nets with invisible (silent) or duplicate-label transitions the two can disagree, so our acceptance number is a proxy; Sect. 6.6 quantifies the disagreement per net class. (iii) The data-driven length cap never binds on D1/D2 (`truncated_traces`=0). On the deep-trace logs it does, and it pays: on D4 the cap alone improves calibration from MAE 0.038 to 0.027 (same weighting, cap as the only change).

Table 1. Benchmark datasets. TLRA $= 1 - |V|/|L|$.

ID	Log	Cases	Events	Variants	Avg. len	TLRA
D1	Sepsis [17]	1,050	15,214	846	14.5	0.19
D2	BPI 2013 Incidents	7,554	65,533	1,511	8.7	0.80
D3	BPI 2017	31,509	1,202,267	15,930	38.2	0.49
D4	BPI 2018	43,809	2,514,266	28,457	57.4	0.35
D5	BPI 2019	251,734	1,595,923	11,973	6.3	0.95

6 Evaluation

This is an algorithmic, quantitative contribution, so we evaluate experimentally. We measure how well each metric agrees with an empirical generalization ground truth across discovery configurations and datasets, and we attribute the proposed metric's behavior to specific design decisions through a measured ablation.

6.1 Experimental setup

Datasets. From a profiled catalog of 20+ public logs we selected five (Table 1) spanning trace depth, variant diversity, scale, and, most deliberately, trace-level representativeness (TLRA, ranging 0.19–0.95), the axis shown to govern the accuracy of generalization estimation [13]. The same evaluation battery runs on every dataset. We present D1 (Sepsis) in full as the primary case, carrying the cross-paradigm comparison and its figures; D2 (BPI 2013 Incidents) corroborates the litmus and the acceptance reading; D3–D5 extend the study to logs of up to 2.5 million events (Sect. 6.5).

Discovery configurations. Eight miners span the construct's poles: a filtered trace model (top-50 variants by frequency; the full trace model is intractable to replay on variant-rich logs), Alpha and Alpha+ [3], Heuristics default and strict [23], Inductive strict and infrequent [15,16], and a flower model. Models are discovered once per (dataset, miner) cell and shared by all methods.

Methods and ground truth. Our metric is M1: ShadowGen as specified in Sect. 4 (release id v2.6-mle; benchmark id M1g). A single ablation baseline (benchmark id M1a) replaces the variable-order model by its 1-gram floor, a directly-follows generator with the same Good–Turing novelty and termination machinery, and isolates what deep context contributes (Sect. 6.5). External baselines are M2–M8 (Sect. 3). Reference metrics are R1 (variant-based 5-fold CV fitness, 3 shuffles), R2 (leave-one-variant-out), and R3 (replay of uniformly random traces, a floor). An acceptance-based variant, R1-accept, measures the fraction of held-out traces replayed perfectly and validates the acceptance reading.

Protocol and agreement criteria. Each cell is evaluated with its method's stochasticity protocol (our metric: 5 iterations of 1,000 shadow traces, mean ± std; seed 42). Agreement with the ground truth is reported on *four criteria*: Pearson (calibration shape), Spearman and Kendall's τ_b (ordering), mean absolute error (MAE, absolute calibration), and discriminative spread (max−min over the six non-degenerate miners). All four are computed over those six miners; the two poles are excluded and reported separately as litmus values. Kendall's τ_b is the fraction of model pairs ordered the same way by the metric and the ground truth, tie-corrected; with six miners a perfect ordering has exact permutation probability $1/6! \approx 0.0014$, which keeps the ranking claims interpretable despite the small sample. Rank correlation alone is too weak a test: as Sect. 6.3 shows, a random-replay baseline also reaches Spearman and Kendall 1.0. The context order was fixed at N=6 via a mutation-rate sweep on BPI 2017 (the realized mutation rate peaks at N=6 and declines beyond, as backoff increasingly rejects sparse contexts; Fig. 3), and N=6 is held fixed across all datasets so the metric is never tuned per dataset.

Compute budget. Every method is granted the same budget of 1 hour of metric time per (dataset, miner) cell. Model discovery is excluded from the budget: models are discovered once per cell, cached, and shared by all methods, so no method pays for discovery twice. The reference metrics (R1, R2, R1-accept) are exempt; they define the ground truth and run to completion. A cell that does not return within the budget is recorded as a sentinel (−1, "exceeds budget") together with the measured evidence, rather than dropped, and a method is not re-run after a timeout. This is a deliberate, uniform protocol rule with two purposes: it keeps the comparison affordable at scale, and, since the working metrics complete a cell in seconds, it turns runtime into a first-class outcome, so that a method's inability to finish is reported as a result rather than silently omitted. Sentinels are excluded from the agreement statistics and shown as distinct cells (not low scores) in the figures. The resulting matrix is complete: of 760 cells (5 datasets × 8 miners × 19 method ids, including the metric's retired development versions, which this report does not discuss), 654 hold measured scores and 106 hold evidence-bearing sentinels; none is missing.

6.2 Feasibility results

Under the compute budget, two published metrics return no score on any cell of any log, which we report as a result. Anti-alignments (M4) ran on the small demo log bundled with its implementation (10 traces, 53 ms), confirming the install works, but produced no score on any miner cell of the 1,050-case Sepsis log: on Alpha+ its Gurobi ILP ran 13.4 h without returning, while on the remaining miners the solver failed outright within seconds. Pattern-based generalization (M8) likewise yielded nothing on every model: it returned no result within its timeout, or failed to initialize its display server. These are not one-off accidents but a reproducible finding: against seconds per cell for every working metric, methods that cannot finish on a small real log are impractical, and the gap

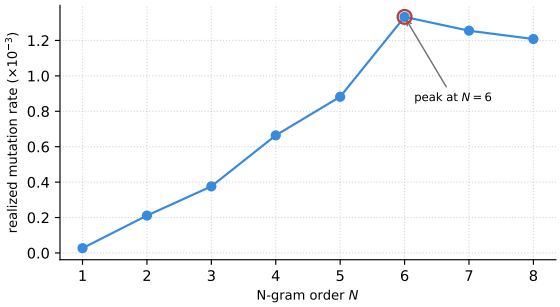

Fig. 3. Realized mutation rate vs. N-gram order on BPI 2017 (the one calibration of the metric). The rate climbs to a peak at N=6 and declines beyond, as Katz backoff increasingly rejects over-sparse high-order contexts; N=6 is then held fixed on every dataset.

widens with dataset size. Entropic relevance (M3) is a DFG-based approximation (Sect. 5.4): it discriminates between miners on all five datasets.

AVATAR (M5) is feasible on the two small logs (two of the 3–5 published sampling runs) and exceeds the budget from D3 upward; we report measured evidence, not a guess. On D3 an attempt consumed the full hour without leaving GAN pretraining. On D4 the training data does not fit: the pipeline was killed by the out-of-memory supervisor at a 16 GB and again at a 24 GB memory cap. D5 is recorded as an evidenced extrapolation from these attempts. A CPU anchor on D1 (100 pretraining epochs plus about 20 of 5,000 adversarial steps in one hour) extrapolates to over 60 CPU-hours for one log, roughly 15× a GPU. Independently of runtime, AVATAR's published configuration caps sequences at 20 events, below the mean trace length of D3 (38.2) and D4 (57.4), so its scores there would truncate the very behavior generalization asks about.

Two further boundary results at scale. The published Entropia `-bgen` tool needs 6 to 10 minutes per D4 cell: five of eight cells return within the hour and the remaining three fail inside the tool, so its D4 row exists because the budget is one hour and not ten minutes. SpeciAL scores six of eight D5 cells; on the Alpha and Alpha+ nets it exceeds the hour, and both cells are recorded as sentinels.

In sum, five of seven published baselines (M2, M3, M5, M6adapted/M6original, M7) produce model-discriminating scores within budget on the two small logs. At scale, M2, M3, M6adapted/M6original, and M7 continue (with the sentinels above), M5 exceeds the budget, and M4/M8 never return a score.

6.3 Cross-paradigm comparison on D1

Tables 2–3 compare paradigms against the cross-validation ground truth. Four findings stand out.

1. Generative and resampling paradigms track held-out fitness; structural and diversity paradigms do not. The most widely available metric (M2) corre-

Table 2. D1 Sepsis: scores per miner, mean ± std where applicable (seven miners of the external comparison). ‡model fitness < 0.5 on the log. †infeasible (no score on any cell).

Method	Alpha‡	Alpha+	Heur.	Heur.S	Ind.Inf	Ind.S	Flower
ShadowGen (ours)	.272±.004	.759±.005	.879±.002	.917±.002	.972±.002	.984±.002	1.00
M2 PM4Py	.913	.919	.841	.900	.880	.903	.913
M5 AVATAR	.340±.020	.562±.008	.746±.001	.704±.002	.751±.002	.535±.009	.397±.007
M6adapted*	.252±.005	.783±.010	.897±.004	.931±.003	.976±.002	.994±.001	1.00
M7 SpeciAL	.799	.750	1.00	.998	.750	.744	.803
M4 Anti-align.†	*infeasible: no score on any cell (Sect. 6.2)*						
M8 Pattern†	*infeasible: no score on any cell*						
R1 5-fold CV	.275±.001	.829±.002	.902±.005	.933±.001	.985±.002	1.00	1.00
R2 LOVO	.306±.075	.775±.177	.870±.051	.917±.044	.981±.053	1.00	1.00
R3 Random	.278±.004	.382±.004	.502±.005	.621±.003	.693±.001	.767±.002	1.00

*M6adapted scores the genuine `bsgen` bootstrap sample by token-replay fitness (a graded model–system recall, the construct the bootstrap paper defines [18]), not the Entropia `-bgen` precision/recall F-measure (Sect. 5.4). †M4 ran 13.4 h on Alpha+ and failed in seconds on the rest; M8 timed out or failed to initialize.

Table 3. Agreement with R1 on D1 (six real miners) and the flower litmus. τ_b is Kendall's tau-b.

Method	Pearson	Spearman	τ_b	MAE	Spread	Flower
ShadowGen (ours)	1.00	1.00	1.00	0.023	0.71	1.00
M2 PM4Py	−0.37	−0.43	−0.20	0.17	0.08	0.91
M5 AVATAR	0.81	0.37	0.33	0.24	0.41	0.40
M6adapted*	1.00	1.00	1.00	0.015	0.74	1.00
M7 SpeciAL	0.12	−0.46	−0.41	0.21	0.26	0.80
R3 Random	0.83	1.00	1.00	0.28	0.49	1.00

lates negatively with the ground truth, compresses seven structurally very different models into a spread of 0.08, and ranks the pathological Alpha model highest. This is direct evidence that visit-frequency counting does not measure held-out generalization.

2. Rank correlation alone is a low bar. The random-replay floor R3 attains Spearman and τ_b of 1.0, because the Sepsis miners differ so strongly in permissiveness that almost any test orders them correctly. Validation must rest on the full set of criteria, which is where ShadowGen and bootstrap (MAE 0.023/0.015, spreads 0.71/0.74) separate from R3 (MAE 0.28, spread 0.49).
3. The flower litmus sorts the field. Pure-acceptance metrics (ShadowGen, M6adapted, R1–R3) score the flower model 1.0, as the construct requires; M2, M7, and especially M5 (0.40) reveal precision/structure contamination. That M6adapted lands in the first group is independent corroboration, not coincidence: the bootstrap metric is itself defined as model–system recall [18], so a separate, published generalization measure also rates the flower model maximally general. M5's low flower score, by contrast, follows from its construct (a harmonic mean of fitness and precision [22], i.e. an F-measure), which also explains its poor rank fidelity on the real miners.

Table 4. Robustness to the cross-validation reference on D1 (six real miners): agreement against both R1 (5-fold) and R2 (leave-one-variant-out, 50/846 variants sampled). The qualitative verdict is identical under either reference.

	vs R1 (5-fold)			vs R2 (LOVO)		
Method	Pear.	τ_b	MAE	Pear.	τ_b	MAE
ShadowGen (ours)	1.00	1.00	0.023	1.00	1.00	0.014
M6adapted*	1.00	1.00	0.015	1.00	1.00	0.019
M2 PM4Py	−0.37	−0.20	0.17	−0.37	−0.20	0.17
M5 AVATAR	0.81	0.33	0.24	0.80	0.33	0.21
M7 SpeciAL	0.12	−0.41	0.21	0.10	−0.41	0.20

Table 5. The same `bsgen` bootstrap sample on D1, scored four ways, vs R1 (six real miners). Only the readings free of precision pass the flower litmus.

Reading of the bootstrap sample	Pearson	τ_b	MAE	Flower	litmus
M6original `-bgen` F1(prec, rec)	0.87	0.20	0.59	0.18	fail
its precision component	0.82	0.20	0.68	0.10	fail
its **recall** component	0.98	0.73	0.11	1.00	pass
M6adapted (token-replay)	1.00	1.00	0.015	1.00	pass

4. Bootstrap is the closest competitor, with a caveat. M6adapted matches our calibration, but it scores with token-replay fitness, the same conformance engine as R1 and our own metric; part of its tight agreement is a shared-ruler effect rather than independent corroboration (Sect. 6.10).

The agreement is robust to the choice of cross-validation reference (Table 4). Against the finer-grained leave-one-variant-out (R2), ShadowGen's ranking is identical (Kendall τ_b=1.0) and its calibration is unchanged on D1 (MAE 0.014); on D2 the ranking again holds while calibration stays strong (Pearson 0.97, MAE 0.048). The verdicts on every baseline are the same under R1 and R2.

The litmus failures also replicate on D2: PM4Py scores the flower model 0.88, AVATAR 0.74, and SpeciAL 0.94, all below the 1.0 the construct requires, while ShadowGen and the references stay at 1.0. Figure 4 plots each metric against the cross-validation ground truth directly; the raw per-cell scores are in Table 2.

6.4 What contaminates a generalization score: a controlled bootstrap study

Bootstrap generalization is published as a generalization metric in its own right [18], so it is worth asking why the number a user typically reads off it (the Entropia `-bgen` F1 of model–system precision and recall) fails the flower litmus. Because one bred `bsgen` sample can be scored several ways, we hold the sampler fixed and vary only the scoring (Table 5).

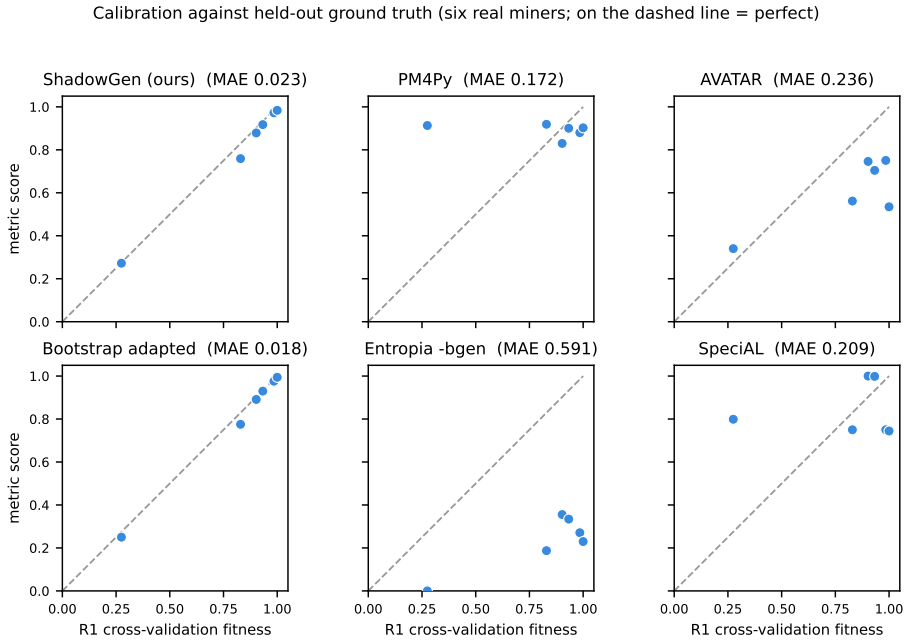

Fig. 4. Per-method calibration against the cross-validation ground truth on D1 (six real miners; each point is one miner, dashed line $y{=}x$). The generative and resampling metrics (ShadowGen, Bootstrap adapted) sit on the line; the published Entropia `-bgen` F-measure of the same bootstrap sample sits far below it (its F1 folds precision in, Sect. 6.4); PM4Py forms a flat high band that ignores the ground-truth gradient: it rates the pathological Alpha model (at R1$\approx$ 0.27) as 0.91; AVATAR and SpeciAL scatter off the line. Reading the vertical axis alone, a metric that spreads its six miners discriminates between them, whereas PM4Py compresses them into a narrow band.

The two readings that include precision fail the litmus (flower 0.18, 0.10) and miscalibrate badly (MAE 0.59, 0.68); the two recall-only readings pass and track R1. The F1 still reaches Pearson 0.87, yet its MAE 0.59 and flower 0.18 expose it, which is why the protocol uses four criteria and not one. So it is the precision component, not the bootstrap sampler, that throws the bootstrap off, and this is why we report it (M6adapted) by token-replay fitness, faithful to the paper's recall construct (Sect. 5.4). Section 6.5 shows the same split at scale: the adaptation stays calibrated on every log, the F1 reading fails everywhere.

6.5 Results at scale: D3–D5

The three large logs multiply the event volume by up to 165× over D1 and change the regime: deeper traces (D4 averages 57.4 events), larger variant spaces (up to 28,457), and both extremes of representativeness (TLRA 0.35–0.95). Table 6

Table 6. Agreement with R1 on all five logs (six real miners each): Pearson / MAE per log, and the flower-litmus range. AVATAR is measured on D1/D2 only (Sect. 6.2); the SpeciAL D5 entry covers the four miners that returned within budget; M4/M8 return no score on any log.

	D1		D2		D3		D4		D5		Flower
Method	Pear.	MAE	Pear.	MAE	Pear.	MAE	Pear.	MAE	Pear.	MAE	
ShadowGen (ours)	.996	.023	.994	.019	.999	.006	.999	.016	.986	.043	1.00 (all)
M6adapted	.998	.018	.978	.034	.999	.006	.993	.020	.998	.014	1.00 (all)
M2 PM4Py	−.35	.172	.41	.184	−.65	.135	−.53	.208	−.33	.229	.88–.98
M7 SpeciAL	.12	.209	.13	.158	.63	.122	.76	.162	−.24	.079	.76–.94
M6original (`-bgen` F1)	.87	.591	.95	.250	.13	.760	.36	.715	.98	.619	.05–.53
R3 Random floor	.83	.281	.97	.116	.70	.265	.88	.176	.84	.196	1.00 (all)

reports the agreement with R1 on all five logs; Fig. 5 plots our metric's 30 real-miner cells against the ground truth.

Six findings.

1. *The calibration holds at scale.* ShadowGen tracks R1 on every log: Pearson 0.986–0.999, MAE 0.006–0.043, spread 0.61–0.76, flower exactly 1.0 everywhere. The ranking is perfect on D1, D2, and D4; on D3 and D5 exactly one adjacent pair swaps (Spearman 0.94). Both swaps are benign: on D3 the pair is separated by 0.0004 in R1 (a tie for practical purposes), on D5 it is the two Alpha variants, both scored far below the field by metric and ground truth alike.
2. *A second generator lands on the same calibration.* The bootstrap adaptation (M6adapted) co-leads on every log (MAE 0.006–0.034, flower 1.0). We read this as corroboration rather than competition: two independently designed synthetic-log generators (bootstrap resampling with crossover breeding; a variable-order N-gram model with calibrated novelty), scored by the same acceptance reading, agree with the ground truth and with each other. The generative paradigm, not one specific generator, carries the result. The practical difference is cost (median 36.9 s against our 14.9 s per model) and the principled novelty injection that resampling lacks (Sect. 3).
3. *The precision contamination of the published reading replicates everywhere.* The Entropia `-bgen` F1 of the same sampler fails the litmus on every log (flower 0.05–0.53) and miscalibrates by an order of magnitude more than the recall-faithful adaptation (MAE 0.250–0.760). The D1 diagnosis of Sect. 6.4 is not a D1 artifact.
4. *PM4Py fails on every log.* Its Pearson is negative on D1, D3, D4, and D5 and 0.41 on D2, its spread never exceeds 0.14, and it fails the flower litmus on all five (0.88–0.98). SpeciAL shows no stable relationship either (Pearson −0.24–0.76) and rates the memorizing trace model at 1.0 on every log, the opposite pole error.
5. *Rank correlation stays a low bar at scale.* The random-replay floor R3 orders the miners nearly perfectly on every log (Spearman 0.87–1.0) while miscalibrating by MAE 0.116–0.281. The four-criteria protocol is what separates the calibrated metrics from the floor, on every dataset.

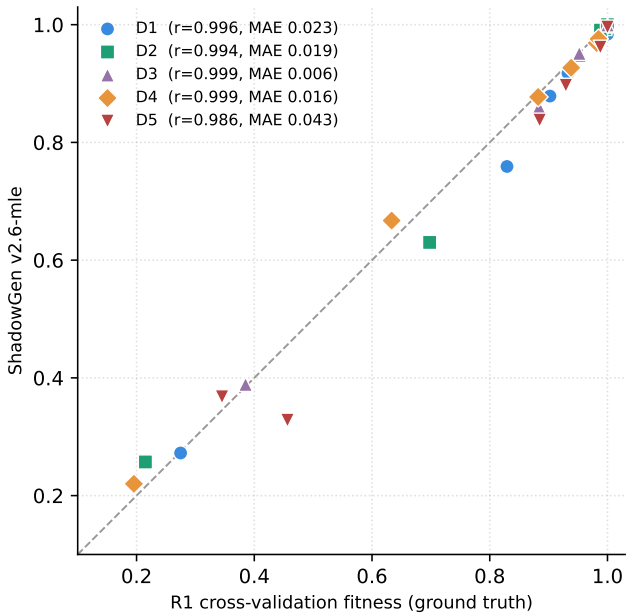

Fig. 5. ShadowGen against the R1 ground truth on the six real miners of every log (30 cells, dashed line $y{=}x$). The calibration measured on D1/D2 carries to logs of 2.5 million events.

6. *Deep context earns its keep on deep logs.* The 1-gram ablation is competitive on the two short-trace, high-TLRA logs (D2 MAE 0.036; on D5 it is even better calibrated, 0.026 against 0.043) but falls behind by 11× on D3 (MAE 0.066 against 0.006), by 2.3× on D4, and by 3× on D1. This confirms the prediction registered in the two-log study: the variable-order model pays off exactly where decisions depend on deep context, and the honest converse is that on shallow repetitive logs a directly-follows generator suffices.

One registered prediction is refuted. Following [13] we predicted that calibration error grows as TLRA falls. It does not: the lowest MAE occurs at TLRA 0.49 (D3, 0.006) and the highest at TLRA 0.95 (D5, 0.043), with no monotone trend across the five logs. At these scales, trace depth and alphabet size govern the difficulty of generalization estimation at least as much as trace-level representativeness does.

6.6 The acceptance reading

Gen_{accept} reports the fraction of shadow traces replayed perfectly, the strict membership-shaped reading. It is best read not as a competing quality score (it is bimodal and rank-uninformative, see below) but as a diagnostic that decomposes

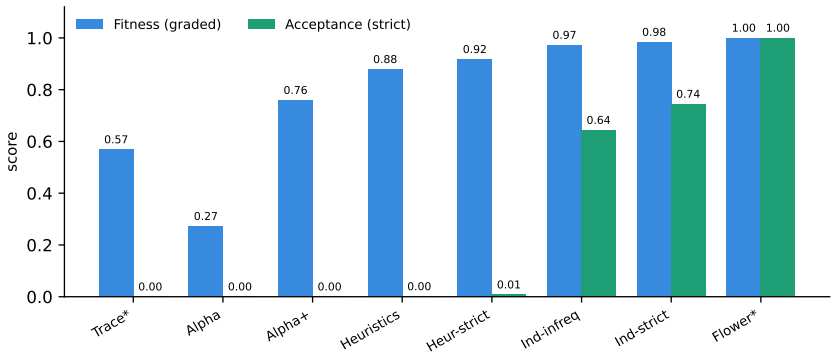

Fig. 6. Two readings of the shadow log on D1 Sepsis: graded fitness (partial credit) vs. strict acceptance (perfect-replay fraction). Acceptance gives the construct's clean poles (Trace 0.00, Flower 1.00), and the gap between the bars exposes partial-credit inflation: e.g. Alpha+ scores 0.76 fitness but 0.00 acceptance, and the trace model 0.57 fitness but 0.00 acceptance.

Table 7. Acceptance reading on D1 (Gen_{accept}) vs the acceptance ground truth (R1-accept).

	Trace	Alpha	Alpha+	Heur.	Heur.S	Ind.Inf	Ind.S	Flower
Gen_{accept}	.000	.000	.000	.001	.008	.644	.744	1.00
R1-accept	.000	.000	.000	.028	.043	.783	.996	1.00

the graded one: Gen_{shadow} is real acceptance plus partial credit, and the per-model gap between the two says how much of a score is real. Acceptance values must be validated against an acceptance-based ground truth (R1-accept: the fraction of held-out real traces replayed perfectly). Against the matching ground truth, Gen_{accept} reaches Pearson 0.992–0.999 and MAE 0.021–0.076 on D1, D2, D3, and D5, and the poles anchor exactly on every log including D4: trace model 0.00, flower model 1.00, in metric and ground truth alike. The D1 detail below carries the argument (Table 7, Fig. 6).

Three observations. The poles anchor exactly: trace model 0.000, flower model 1.000, in both metric and ground truth, on every dataset. These are the clean 0/1 bounds that graded fitness cannot provide. Near-zero acceptances are true model properties, not metric failures: on D1 the ground truth itself shows Alpha, Alpha+, and both Heuristics models strictly accepting (almost) no unseen real traces (e.g., Alpha+ 0.000, Heuristics 0.028); only the Inductive family genuinely accepts novel behavior. Alpha+'s respectable graded score (0.83 under R1) is thus pure partial credit, which the acceptance reading exposes in one line, and the per-model gap between Gen_{shadow} and Gen_{accept} is itself diagnostic. Caveats: strict acceptance is bimodal and sensitive to net soundness (which is why it accompanies rather than replaces the graded score), rank correlation is

uninformative under mass ties at zero (use Pearson/MAE), and Gen_{accept} underestimates R1-accept in the mid-range on D1 because shadow traces (containing novel recombinations and mutations) are deliberately harder than held-out real traces.

D4 exposes a depth limit of the strict reading, and we report it as a finding rather than smoothing it away. On D4 (mean trace length 57.4) even held-out real traces barely replay perfectly: the ground truth's largest value is 0.30 (Inductive-strict), and our shadow traces, which deliberately contain novelty, almost never do (0.0002 on that cell). Every other D4 cell is near zero on both axes, so only two mid-range cells carry signal, Pearson is uninformative on D4, and MAE stays at 0.057. The cause is mechanical: one deviation anywhere in a 57-event trace breaks a perfect replay, so strict acceptance saturates toward 0 as traces deepen. The poles still anchor exactly. This is a property of membership-shaped generalization at depth, not of our generator, and it is one more reason the graded score is the headline and acceptance the diagnostic.

We also quantified the proxy error flagged in Sect. 5.5: on D1 shadow traces (1,000 per net), the token-replay flag agrees with cost-zero alignments exactly (1.0000) on the Inductive-strict, Flower, and Heuristics nets, and reaches 0.804 on Inductive-infrequent, where all 196 disagreements are one-directional over-accepts of the replay flag (invisible-transition leniency). Mid-range acceptance values on that net class are therefore upper bounds; the poles and the validation above are unaffected.

6.7 The generator premise, tested directly

The central premise of the metric is that the shadow log resembles valid future behavior. The earlier draft argued this constructively; the claim is now measured. For every R1 fold partition (3 shuffles $\times$ 5 folds per log), we train the generator on the train fold only, generate 1,000 traces, and count how many are exact activity-sequence matches of variants in the held-out test fold. The generator deduplicates against its own training variants, and train and test folds share no variants, so a hit cannot be memorization: it is a real future variant the generator produced without ever seeing it. Two baselines calibrate the number: the 1-gram ablation, and uniformly random traces over the training alphabet and length distribution.

Three readings of Table 8. First, the premise holds: on D5 a quarter of the generated traces are unseen real future variants, and random traces are never one (the only nonzero random cell is 0.01% on D2's four-activity alphabet). Second, deep context is what produces the realism where it matters: on D3 the full model hits 8.6% of the time and the 1-gram ablation never does. Third, the exact-match criterion is length-limited by construction: a hit must reproduce every step, so the rate falls steeply with trace depth. D4 at 57 events per trace is near zero for that reason alone; its graded calibration (MAE 0.016) shows the generator models D4 well, and the drop mirrors the strict-acceptance saturation of Sect. 6.6. Exact matches are therefore a conservative lower bound on the

Table 8. Generator premise: share of generated traces that exactly match held-out real variants (mean over 15 folds, in %), and the share of the held-out variant set reached (coverage, in %).

Log	ShadowGen	1-gram	Random	Coverage
D1	1.1	0.4	0.0	4.8
D2	16.6	10.9	0.01	10.6
D3	8.6	0.0	0.0	1.4
D4	0.09	0.0	0.0	0.01
D5	25.9	5.5	0.0	1.7

shadow log's realism, and even this bound is orders of magnitude above chance.

6.8 Runtime

The metric evaluates one cell ($5 \times 1{,}000$ traces) in 2–7 seconds on the small logs and at a median of 14.9 seconds per model over all five logs; the worst cell is 22 minutes (D4, Inductive-strict, where replay on the discovered net dominates). The bootstrap adaptation has the same shape at 2.5× the cost (median 36.9 s). The references are of a different order: R1 has a median of 19 minutes per model on D4 and a worst cell of 5.7 hours, and leave-one-variant-out reaches 46 hours on a single cell. AVATAR sits at the budget boundary (Sect. 6.2). Figure 7 sets accuracy against time over the whole benchmark. Among methods that agree with the ground truth, ours is the fastest, and it stays defined where the ground truth is not: it scores a model artifact in hand, without rediscovery (Sect. 4.6).

6.9 Discussion

On five real-life logs spanning 15 thousand to 2.5 million events, the methods that confront a model with resampled or synthesized behavior (our metric, bootstrap) agree with the cross-validation ground truth in both rank and absolute value, whereas methods that inspect structure or aggregate diversity (PM4Py, SpeciAL) do not, on any log. This supports the project's core hypothesis: generalization is a claim about unseen behavior and is therefore best measured by probing models with plausible unseen behavior, instead of analysing the existing structure. That two independently designed generators land on the same calibration strengthens the point: the paradigm carries the result, and our generator adds interpretability, principled novelty, and a 2.5× speed advantage on top of it. Our metric reaches this agreement at interactive runtime with interpretable internals (per-context novelty, mutation flags, the openness profile, the acceptance reading) that neural and bootstrap aggregates do not expose. The honest claim is therefore not "best metric" but "matches the strongest baseline's calibration while probing genuinely unseen behavior that resampling cannot reach, and exposing why a model scored as it did, at comparable cost and with no external toolchain."

Fig. 7. Speed against accuracy over all five logs: median time per model (log scale) versus mean of the per-log MAEs to R1; lower-left is better. Whiskers show the spread the aggregates hide: horizontal, min to max cell runtime; vertical, min to max per-log MAE. ShadowGen and the bootstrap adaptation are alone in the accurate corner; PM4Py is fast but never accurate; the Entropia `-bgen` F-measure is slow and far off; AVATAR (D1/D2 only) is plotted at its budget anchor without a time whisker, since its runtime at scale is a bound, not a measurement. M4 and M8 return no score on any log.

6.10 Threats to validity

Breadth: the study covers five logs and eight miners per log, but every correlation is computed over six non-degenerate miners, a deliberately small, interpretable sample; dataset-specific structure can still favor particular methods, and the two-log external comparison of AVATAR limits its verdict to the regime where it runs.

Shared measurement core: the metric, R1, R3, and M6adapted all score via token replay, so part of their mutual agreement may reflect a shared fitness notion rather than shared generalization semantics. R1's per-fold rediscovery limits but does not eliminate this, and the acceptance reading was already validated against a separately computed ground truth (R1-accept).

Ground truth and representativeness: our validation rests on agreement with variant-based hold-out (R1/R2). We regard this as the strongest available empirical reference for generalization, because its test traces are real held-out behavior and so are provably valid, where a shadow trace is only plausible by construction. The close agreement between the two is therefore evidence that the shadow log behaves like genuine future behavior. ShadowGen also reaches this agreement while scoring a model in hand, which R1 cannot: R1 evaluates a discovery algorithm by rediscovering on log subsets, not a model artifact. Both are computed

from the log, so both are only as faithful to the true system as the log is representative of it. We treat that as a responsibility of the data-gathering stage rather than of the metric: any analysis built on a log, including a model designed by hand, is bounded in validity by how representatively the log samples the underlying process. The registered prediction that our calibration error grows as representativeness falls was tested and refuted (Sect. 6.5); the bound is real, but TLRA alone does not govern it.

Generator validity: the premise that the shadow log resembles valid future behavior is now tested directly (Sect. 6.7): up to 25.9% of generated traces are exact held-out real variants, against 0% for random traces. The residual threat is that exact matching is a lower-bound criterion: on deep-trace logs it cannot certify realism (D4: 0.09%), where the premise is supported only indirectly by the graded calibration.

Adapted and incomplete baselines: M6adapted uses token-replay fitness in place of the Entropia precision/recall F-measure, and its sampler is our reimplementation from the paper, validated on D1 (Sect. 5.4); M3 uses a DFG approximation (Sect. 5.4) rather than a proper SDFA conversion; M5 has two of 3–5 planned sampling runs on D1/D2 and evidence-based sentinels beyond; M7's D5 cells use a newer package version than D1–D4 (Sect. 5.4); R2 sampled 50 of 846 variants on D1, and the D4 Inductive-strict cell of R2 is sampled at 50 of 28,457 variants (a full leave-one-variant-out there would need 28,457 rediscoveries).

Calibration provenance: N=6 was selected on one log via a proxy criterion (mutation rate), and the mutated-trace share varies strongly across logs ($\sim$ 4% on BPI 2017 vs. $\sim$ 50% on Sepsis), so proposal-distribution effects are dataset-dependent by construction.

Replay reproducibility, measured: PM4Py's token replay breaks ties by element iteration order on nets with duplicate labels or silent transitions, and no seed pins it. We measured the drift across independent processes: the trace model moves by up to 0.02, Alpha+ by up to 0.06 (on D4), and every other cell reproduces exactly. The four-criteria agreement is essentially unaffected (the worst-case D4 MAE moves from 0.015 to 0.016), and all discovery-side nondeterminism is pinned (fixed hash seed, models discovered once and cached). The acceptance proxy itself is quantified in Sect. 6.6.

7 Conclusion

Contribution and findings. We presented ShadowGen, a generative N-gram metric for process-model generalization, together with a strict construct definition it implements and a benchmark that validates generalization metrics against variant-based cross-validation under a four-criterion protocol with both construct poles included. The central result is that ShadowGen tracks the empirical ground truth on all five real-life logs, from 15 thousand to 2.5 million events: it reproduces the cross-validation ranking (at most one adjacent swap per log) and is calibrated to it (Pearson 0.986–0.999, MAE 0.006–0.043) at a median of 15 seconds per model, and it does so while scoring a model in hand, where the

cross-validation reference cannot. The benchmark adds findings of independent value: the field's most accessible metric (PM4Py) is anti-correlated with empirical generalization on four of five logs and fails the flower litmus on all five; two published metrics return no score on any log under a uniform one-hour budget; rank correlation alone is too weak a validation criterion on every log; an independently designed bootstrap generator, scored by the same acceptance reading, lands on the same calibration, corroborating the generative paradigm itself; the strict acceptance reading anchors the construct's poles exactly on every log and exposes both partial-credit inflation and its own depth limit; and the generator premise is confirmed directly, with up to 25.9% of generated traces being exact held-out real variants.

Limitations. The metric and much of its cross-validation reference share a token-replay core, so part of their agreement reflects a shared conformance notion rather than shared generalization semantics; the context order N=6 is calibrated on a single log via a proxy; several baselines are adapted, degraded, or incomplete; the strict acceptance reading saturates on deep-trace logs and its membership test is a replay proxy with a measured one-sided error on one net class (0.804 agreement on Inductive-infrequent).

Future work. The natural next steps are an alignment-based acceptance reading that removes the replay proxy, restoring entropic relevance through a per-model stochastic-automaton conversion, and an AVATAR comparison under a GPU budget, so the second generative paradigm can be judged at scale. The metric itself is feature-frozen, with the `mle` mode as the default and the name ShadowGen reflecting its purely generative design.

Declaration on the Use of AI-Based Tools

In preparing this work, we used AI-based assistants (large-language-model and coding tools) for source-code drafting, figure generation, and language editing; all content was reviewed and verified by the authors, who take full responsibility for it.

References

1. van der Aalst, W.M.P.: Process Mining: Data Science in Action. Springer, Heidelberg, 2nd edn. (2016). https://doi.org/10.1007/978-3-662-49851-4
2. van der Aalst, W.M.P.: Relating process models and event logs — 21 conformance propositions. In: Proc. Int. Workshop on Algorithms & Theories for the Analysis of Event Data (ATAED 2018). CEUR Workshop Proceedings, vol. 2115, pp. 56–74 (2018)
3. van der Aalst, W.M.P., Weijters, A.J.M.M., Maruster, L.: Workflow mining: Discovering process models from event logs. IEEE Transactions on Knowledge and Data Engineering **16**(9), 1128–1142 (2004). https://doi.org/10.1109/TKDE.2004.47

4. Alkhammash, H., Polyvyanyy, A., Moffat, A., García-Bañuelos, L.: Entropic relevance: A mechanism for measuring stochastic process models discovered from event data. Information Systems **107**, 101922 (2022). https://doi.org/10.1016/j.is.2021.101922
5. Augusto, A., Conforti, R., Dumas, M., La Rosa, M., Maggi, F.M., Marrella, A., Mecella, M., Soo, A.: Automated discovery of process models from event logs: Review and benchmark. IEEE Transactions on Knowledge and Data Engineering **31**(4), 686–705 (2019). https://doi.org/10.1109/TKDE.2018.2841877
6. Berti, A., van Zelst, S.J., van der Aalst, W.M.P.: Process mining for Python (PM4Py): Bridging the gap between process- and data science. In: Proceedings of the ICPM Demo Track 2019. CEUR Workshop Proceedings, vol. 2374, pp. 13–16 (2019)
7. Buijs, J.C.A.M., van Dongen, B.F., van der Aalst, W.M.P.: Quality dimensions in process discovery: The importance of fitness, precision, generalization and simplicity. International Journal of Cooperative Information Systems **23**(1), 1440001 (2014). https://doi.org/10.1142/S0218843014400012
8. van Dongen, B.F., Carmona, J., Chatain, T.: A unified approach for measuring precision and generalization based on anti-alignments. In: Business Process Management (BPM 2016). LNCS, vol. 9850, pp. 39–56. Springer (2016). https://doi.org/10.1007/978-3-319-45348-4_3
9. Gale, W.A., Sampson, G.: Good–Turing frequency estimation without tears. Journal of Quantitative Linguistics **2**(3), 217–237 (1995). https://doi.org/10.1080/09296179508590051
10. Janssenswillen, G., Depaire, B.: Towards confirmatory process discovery: Making assertions about the underlying system. Business & Information Systems Engineering **61**(6), 713–728 (2019). https://doi.org/10.1007/s12599-019-00610-6
11. Janssenswillen, G., Donders, N., Jouck, T., Depaire, B.: A comparative study of existing quality measures for process discovery. Information Systems **71**, 1–15 (2017). https://doi.org/10.1016/j.is.2017.06.002
12. Kabierski, M., Richter, M., Weidlich, M.: Addressing the log representativeness problem using species discovery. In: Proceedings of the 5th International Conference on Process Mining (ICPM 2023). pp. 65–72. IEEE (2023). https://doi.org/10.1109/ICPM60904.2023.10271952
13. Karunaratne, A., Polyvyanyy, A., Moffat, A.: The role of log representativeness in estimating generalization in process mining. In: Proceedings of the 6th International Conference on Process Mining (ICPM 2024). IEEE (2024)
14. Katz, S.M.: Estimation of probabilities from sparse data for the language model component of a speech recognizer. IEEE Transactions on Acoustics, Speech, and Signal Processing **35**(3), 400–401 (1987). https://doi.org/10.1109/TASSP.1987.1165125
15. Leemans, S.J.J., Fahland, D., van der Aalst, W.M.P.: Discovering block-structured process models from event logs — A constructive approach. In: Application and Theory of Petri Nets and Concurrency (PETRI NETS 2013). LNCS, vol. 7927, pp. 311–329. Springer (2013). https://doi.org/10.1007/978-3-642-38697-8_17
16. Leemans, S.J.J., Fahland, D., van der Aalst, W.M.P.: Discovering block-structured process models from event logs containing infrequent behaviour. In: Business Process Management Workshops (BPM 2013). LNBIP, vol. 171, pp. 66–78. Springer (2014). https://doi.org/10.1007/978-3-319-06257-0_6
17. Mannhardt, F.: Sepsis cases — event log (2016). https://doi.org/10.4121/uuid:915d2bfb-7e84-49ad-a286-dc35f063a460

18. Polyvyanyy, A., Moffat, A., García-Bañuelos, L.: Bootstrapping generalization of process models discovered from event data. In: Advanced Information Systems Engineering (CAiSE 2022). LNCS, vol. 13295, pp. 36–54. Springer (2022). https://doi.org/10.1007/978-3-031-07472-1_3
19. Reißner, D., Armas-Cervantes, A., Conforti, R., Dumas, M., Fahland, D., La Rosa, M.: Scalable alignment of process models and event logs: An approach based on automata and S-components. Information Systems **94**, 101561 (2020). https://doi.org/10.1016/j.is.2020.101561
20. Rozinat, A., van der Aalst, W.M.P.: Conformance checking of processes based on monitoring real behavior. Information Systems **33**(1), 64–95 (2008). https://doi.org/10.1016/j.is.2007.07.001
21. Syring, A.F., Tax, N., van der Aalst, W.M.P.: Evaluating conformance measures in process mining using conformance propositions. In: Transactions on Petri Nets and Other Models of Concurrency XIV, LNCS, vol. 11790, pp. 192–221. Springer (2019). https://doi.org/10.1007/978-3-662-60651-3_8
22. Theis, J., Darabi, H.: Adversarial system variant approximation to quantify process model generalization. IEEE Access **8**, 194410–194427 (2020). https://doi.org/10.1109/ACCESS.2020.3033450
23. Weijters, A.J.M.M., van der Aalst, W.M.P., de Medeiros, A.K.A.: Process mining with the HeuristicsMiner algorithm. Tech. Rep. WP 166, Eindhoven University of Technology (2006)